LLM GATEWAY

Connect Claude Code to an LLM gateway

Point Claude Code at your organization's LLM gateway. Check whether your admin already configured it, or set the base URL and credential yourself for the CLI, VS Code, GitHub Actions, and the Agent SDK, then verify the connection and fix gateway errors.

An **LLM gateway** is a proxy your organization runs between Claude Code and the model provider. When your organization uses one, Claude Code authenticates to the gateway with a credential your organization issues instead of your personal claude.ai login.

This page is for developers running Claude Code through a gateway their organization operates. It covers two paths: **checking whether your administrator already configured it for you**, and **configuring it yourself** when they haven't.

- To deploy a gateway for your organization, see **Roll out an LLM gateway**
- For what Claude Code sends to a gateway, see the **gateway protocol reference**

Check for an existing configuration

Administrators can distribute the gateway address and credential through **managed settings**, device management, or an `apiKeyHelper`, so Claude Code picks them up at startup with nothing for you to set. To check whether your organization already did this:

1 Start Claude Code

Run `claude`. If it opens to the login screen instead of a session, no gateway credential was distributed; **configure it yourself** below.

2 Check the Status tab

If Claude Code started a session without showing the login screen, run `/status`, open the **Status** tab, and check two lines:

- `Anthropic base URL`: this line only appears when a gateway address is set. If it isn't there, Claude Code isn't pointed at the gateway; **configure it yourself** below.
- `Auth token` or `API key`: a line naming `ANTHROPIC_AUTH_TOKEN`, `ANTHROPIC_API_KEY`, or an `apiKeyHelper` confirms a gateway credential is active. A `Login method` line naming a claude.ai account instead means the credential wasn't distributed; **set it yourself**.

3 **Send a test message**

Close the `/status` menu and send any prompt in Claude Code. A normal response from Claude, with no error, confirms the gateway connection works.

If both lines in the `/status` menu look right but the message to Claude fails, see the **troubleshooting table**.

Configure Claude Code yourself

To configure Claude Code for the gateway yourself, you need from your gateway team:

- The gateway's base URL
- A credential: a key or token string, or a command that fetches one
  - If your gateway team didn't say which kind of credential it is, the **credential variable section** below covers what to try

The sections below cover the configuration in order:

- **Set the credential variable** and **set the base URL**: the two variables every gateway connection needs
- **Verify the connection**: confirm it works before persisting anything
- **Configure each surface**: if you are using a surface besides the Claude Code CLI, such as VS Code, see how to configure it with your gateway credentials
- **Additional configuration**: variables some gateways need beyond the base URL and credential, such as a custom header, a credential helper, model discovery, or a provider-format base URL. Set these only if your administrator named them

Set the credential variable

To authenticate Claude Code to the gateway, set your credential in an environment variable. Which variable depends on what your gateway team told you:

Set the credential in	Use when
`ANTHROPIC_AUTH_TOKEN`	Your gateway team said “bearer token” or “Authorization header”
`ANTHROPIC_API_KEY`	Your gateway team said “API key” or “x-api-key”
`apiKeyHelper`	The credential rotates or comes from a vault

If you weren’t told which kind, use `ANTHROPIC_AUTH_TOKEN`; the **verification request** below shows how to tell if you need to switch.

Set the base URL and credential

Set the gateway’s base URL and the credential variable you picked above as environment variables. The examples use `ANTHROPIC_AUTH_TOKEN`; swap it for `ANTHROPIC_API_KEY` if that’s **the variable you picked**. You can set them **in your shell**, which lasts for one terminal session, or **in a Claude Code settings file**, which persists everywhere Claude Code runs.

For your first connection, start with shell exports and run the **verification request** before moving the values to a settings file.

Set as shell environment variables

Replace the values with the ones your gateway team gave you:

Bash or Zsh

```
export ANTHROPIC_BASE_URL=https://llm-gateway.example.com
export ANTHROPIC_AUTH_TOKEN=sk-gateway-key
```

PowerShell

```
$env:ANTHROPIC_BASE_URL = "https://llm-gateway.example.com"
$env:ANTHROPIC_AUTH_TOKEN = "sk-gateway-key"
```

```
curl -X POST "$ANTHROPIC_BASE_URL/v1/messages" \
  -H "Authorization: Bearer $ANTHROPIC_AUTH_TOKEN" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

```
Invoke-RestMethod -Method Post -Uri
"$env:ANTHROPIC_BASE_URL/v1/messages" `
  -Headers @{ "Authorization" = "Bearer
$env:ANTHROPIC_AUTH_TOKEN"; "anthropic-version" = "2023-06-01" }
`
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```
  const result = query({
    prompt: "...",
    options: {
      env: {
        ...process.env,
        ANTHROPIC_BASE_URL: "https://llm-gateway.example.com",
        ANTHROPIC_AUTH_TOKEN: process.env.GATEWAY_KEY,
      },
    },
```

```
})
```

```
export ANTHROPIC_CUSTOM_HEADERS="X-Org-Route: prod"
```

```
$env:ANTHROPIC_CUSTOM_HEADERS = "X-Org-Route: prod"
```

For example, a script that reads from a vault:

```
#!/bin/bash
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference its path in `~/.claude/settings.json`:

```
{
  "apiKeyHelper": "~/bin/get-gateway-key.sh"
}
```

For example, a script that reads from a vault:

```
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference the PowerShell invocation in `%USERPROFILE%\.claude\settings.json`, escaping the backslashes in the JSON string:

```
{
  "apiKeyHelper": "powershell -NoProfile -File C:\\scripts\\get-
gateway-key.ps1"
}
```

```
export ANTHROPIC_BEDROCK_BASE_URL=https://llm-
gateway.example.com/bedrock
export CLAUDE_CODE_SKIP_BEDROCK_AUTH=1
export CLAUDE_CODE_USE_BEDROCK=1
```

```
$env:ANTHROPIC_BEDROCK_BASE_URL = "https://llm-
gateway.example.com/bedrock"
$env:CLAUDE_CODE_SKIP_BEDROCK_AUTH = "1"
$env:CLAUDE_CODE_USE_BEDROCK = "1"
```

```
export ANTHROPIC_VERTEX_BASE_URL=https://llm-
gateway.example.com/vertex
export ANTHROPIC_VERTEX_PROJECT_ID=your-gcp-project-id
export CLAUDE_CODE_SKIP_VERTEX_AUTH=1
export CLAUDE_CODE_USE_VERTEX=1
export CLOUD_ML_REGION=us-east5
```

```
$env:ANTHROPIC_VERTEX_BASE_URL = "https://llm-
gateway.example.com/vertex"
$env:ANTHROPIC_VERTEX_PROJECT_ID = "your-gcp-project-id"
$env:CLAUDE_CODE_SKIP_VERTEX_AUTH = "1"
$env:CLAUDE_CODE_USE_VERTEX = "1"
$env:CLOUD_ML_REGION = "us-east5"
```

```
export ANTHROPIC_FOUNDRY_BASE_URL=https://llm-
gateway.example.com/foundry
export ANTHROPIC_FOUNDRY_API_KEY=sk-gateway-key
export CLAUDE_CODE_USE_FOUNDRY=1
```

```
$env:ANTHROPIC_FOUNDRY_BASE_URL = "https://llm-
gateway.example.com/foundry"
$env:ANTHROPIC_FOUNDRY_API_KEY = "sk-gateway-key"
$env:CLAUDE_CODE_USE_FOUNDRY = "1"
```

```
export ANTHROPIC_AWS_BASE_URL=https://llm-
gateway.example.com/anthropic-aws
export ANTHROPIC_AWS_WORKSPACE_ID=wrkspc_01ABCDEFGHIJKLMN
export CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH=1
export CLAUDE_CODE_USE_ANTHROPIC_AWS=1
```

```
$env:ANTHROPIC_AWS_BASE_URL = "https://llm-
gateway.example.com/anthropic-aws"
$env:ANTHROPIC_AWS_WORKSPACE_ID = "wrkspc_01ABCDEFGHIJKLMN"
$env:CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH = "1"
$env:CLAUDE_CODE_USE_ANTHROPIC_AWS = "1"
```

Shell exports apply only to that terminal session and programs started from it; an editor launched from the dock or Start menu won't see them. To make them persist across new terminals, add the same lines to your shell profile, such as `~/.zshrc`, `~/.bashrc`, or your PowerShell `$PROFILE`, or use a settings file instead.

Set in a settings file

To make the configuration apply everywhere Claude Code runs without depending on your shell, set the variables in the `env` block of a **settings file**. Settings files have different scopes:

- `~/.claude/settings.json` applies to all your projects. On Windows the path is `%USERPROFILE%\.claude\settings.json`
- `.claude/settings.local.json` applies to one project. Claude Code adds it to your gitignore when it creates the file; if you create it yourself, add it to your gitignore manually first so you don't accidentally commit your credential

> ⚠ Don't put the credential in a project's `.claude/settings.json`. That file is committed and shared with everyone who clones the repository.

The `env` block looks the same in either file:

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "https://llm-gateway.example.com",
    "ANTHROPIC_AUTH_TOKEN": "sk-gateway-key"
  }
}
```

When both a shell export and a settings-file `env` block set the same variable, the settings-file value applies. Run `/status` to see which base URL and credential source Claude Code is using.

Verify the connection

With the variables exported in your shell, send a one-token request to the gateway directly. This confirms the URL and credential work before you open Claude Code, so a failure points at the gateway rather than your configuration. The commands below read the shell variables, so they need the **shell exports** even if you also put the values in a settings file.

Bash or Zsh

```
export ANTHROPIC_BASE_URL=https://llm-gateway.example.com
export ANTHROPIC_AUTH_TOKEN=sk-gateway-key
```

PowerShell

```
$env:ANTHROPIC_BASE_URL = "https://llm-gateway.example.com"
$env:ANTHROPIC_AUTH_TOKEN = "sk-gateway-key"
```

```
curl -X POST "$ANTHROPIC_BASE_URL/v1/messages" \
  -H "Authorization: Bearer $ANTHROPIC_AUTH_TOKEN" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

```
Invoke-RestMethod -Method Post -Uri
"$env:ANTHROPIC_BASE_URL/v1/messages" `
  -Headers @{ "Authorization" = "Bearer
$env:ANTHROPIC_AUTH_TOKEN"; "anthropic-version" = "2023-06-01" }
`
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```
  const result = query({
    prompt: "...",
    options: {
      env: {
        ...process.env,
        ANTHROPIC_BASE_URL: "https://llm-gateway.example.com",
        ANTHROPIC_AUTH_TOKEN: process.env.GATEWAY_KEY,
      },
    },
  })
```

```
export ANTHROPIC_CUSTOM_HEADERS="X-Org-Route: prod"
```

```
$env:ANTHROPIC_CUSTOM_HEADERS = "X-Org-Route: prod"
```

For example, a script that reads from a vault:

```
#!/bin/bash
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference its path in `~/.claude/settings.json`:

```
{
  "apiKeyHelper": "~/bin/get-gateway-key.sh"
}
```

For example, a script that reads from a vault:

```
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference the PowerShell invocation in `%USERPROFILE%\.claude\settings.json`, escaping the backslashes in the JSON string:

```
{
  "apiKeyHelper": "powershell -NoProfile -File C:\\scripts\\get-
gateway-key.ps1"
}
```

```
export ANTHROPIC_BEDROCK_BASE_URL=https://llm-
gateway.example.com/bedrock
export CLAUDE_CODE_SKIP_BEDROCK_AUTH=1
export CLAUDE_CODE_USE_BEDROCK=1
```

```
$env:ANTHROPIC_BEDROCK_BASE_URL = "https://llm-
gateway.example.com/bedrock"
$env:CLAUDE_CODE_SKIP_BEDROCK_AUTH = "1"
$env:CLAUDE_CODE_USE_BEDROCK = "1"
```

```
export ANTHROPIC_VERTEX_BASE_URL=https://llm-
gateway.example.com/vertex
export ANTHROPIC_VERTEX_PROJECT_ID=your-gcp-project-id
export CLAUDE_CODE_SKIP_VERTEX_AUTH=1
export CLAUDE_CODE_USE_VERTEX=1
export CLOUD_ML_REGION=us-east5
```

```
$env:ANTHROPIC_VERTEX_BASE_URL = "https://llm-
gateway.example.com/vertex"
$env:ANTHROPIC_VERTEX_PROJECT_ID = "your-gcp-project-id"
$env:CLAUDE_CODE_SKIP_VERTEX_AUTH = "1"
$env:CLAUDE_CODE_USE_VERTEX = "1"
$env:CLOUD_ML_REGION = "us-east5"
```

```
export ANTHROPIC_FOUNDRY_BASE_URL=https://llm-
gateway.example.com/foundry
export ANTHROPIC_FOUNDRY_API_KEY=sk-gateway-key
export CLAUDE_CODE_USE_FOUNDRY=1
```

```
$env:ANTHROPIC_FOUNDRY_BASE_URL = "https://llm-
gateway.example.com/foundry"
$env:ANTHROPIC_FOUNDRY_API_KEY = "sk-gateway-key"
$env:CLAUDE_CODE_USE_FOUNDRY = "1"
```

```
export ANTHROPIC_AWS_BASE_URL=https://llm-
gateway.example.com/anthropic-aws
export ANTHROPIC_AWS_WORKSPACE_ID=wrkspc_01ABCDEFGHIJKLMN
export CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH=1
export CLAUDE_CODE_USE_ANTHROPIC_AWS=1
```

```
$env:ANTHROPIC_AWS_BASE_URL = "https://llm-
gateway.example.com/anthropic-aws"
$env:ANTHROPIC_AWS_WORKSPACE_ID = "wrkspc_01ABCDEFGHIJKLMN"
$env:CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH = "1"
$env:CLAUDE_CODE_USE_ANTHROPIC_AWS = "1"
```

If your gateway expects keys in the `x-api-key` header, replace the `Authorization` header with `x-api-key: $ANTHROPIC_API_KEY` in the Bash command, or the `"Authorization"` hashtable entry with `"x-api-key" = "$env:ANTHROPIC_API_KEY"` in the PowerShell command.

A JSON response that starts with `{"id":"msg_` and includes a `"content":[...]` field means the gateway is reachable and the credential works. An error naming an unknown model still proves the URL and credential work, since the gateway authenticated the request before rejecting the model name; you don't need to find a model your gateway serves for this test. A `401` means the credential was rejected: if you guessed the variable, switch to the other one and re-export.

Confirm in Claude Code

Start `claude` from the same shell so it inherits the exports, send a message, and run `/status`.

On the **Status** tab, the `Anthropic base URL` line should show your gateway address, which confirms requests are routing there; if the line isn't there, the variable didn't reach the session. An `Auth token` or `API key` line naming the variable you set confirms the gateway credential is active rather than a saved claude.ai login.

If the message fails, or `/status` doesn't show the gateway URL, see the **troubleshooting table** below.

How the credential variable maps to a header

Each variable sends the credential in a different HTTP header: `ANTHROPIC_AUTH_TOKEN` in `Authorization: Bearer`, `ANTHROPIC_API_KEY` in `x-api-key`, and `apiKeyHelper` in both. A credential in the wrong variable reaches the gateway in a header it doesn't read, and the request fails with `401`. If the verification request returned `401`, switch to the other variable and try again.

Conflicts with an existing login

A gateway credential variable takes precedence over a saved claude.ai login or Console key. Your claude.ai login stays saved and unused while the variable is set; unset the variable and Claude Code goes back to it. With `ANTHROPIC_AUTH_TOKEN`, the variable takes precedence immediately. With `ANTHROPIC_API_KEY`, you are prompted once in interactive mode to approve the key before it takes over.

Run `/status` to confirm which credential source is active. If startup shows an auth-conflict warning naming two sources, see the first row of the **troubleshooting table** for which one to drop. To clear a saved login so only the gateway credential remains, run `/logout`.

Configure each surface

The CLI reads the environment variables and settings files above. The other surfaces are the VS Code extension, the desktop app, GitHub Actions, the Agent SDK, and the cloud surfaces such as Slack and the web; the sections below cover whether those settings reach each one.

VS Code extension

Set the gateway variables for the **VS Code extension** in `claudeCode.environmentVariables`, in VS Code's own user settings opened with the **Preferences: Open User Settings (JSON)** command. The extension checks credentials from this setting before launching, so it's the reliable place for the gateway credential; values in `~/.claude/settings.json` reach the spawned process but not the extension's own login check.

```
{
  "claudeCode.environmentVariables": [
    { "name": "ANTHROPIC_BASE_URL", "value": "https://llm-
gateway.example.com" },
    { "name": "ANTHROPIC_AUTH_TOKEN", "value": "sk-gateway-key" }
  ]
}
```

Desktop app

The desktop app reads gateway routing from an **administrator-distributed configuration**, not from `ANTHROPIC_BASE_URL` or `settings.json`. If your organization has distributed it, the desktop app routes through the gateway with no setup on your part; if not, use the terminal CLI or VS Code

extension for gateway sessions. Administrators distribute the configuration as described in the **organization rollout**.

If the desktop app shows `Gateway was unreachable`, the app couldn't reach the configured base URL at startup; check the URL and network path with the **curl test above**.

GitHub Actions

Claude Code GitHub Actions reads `ANTHROPIC_BASE_URL` and `ANTHROPIC_CUSTOM_HEADERS` from the workflow's `env` block. Pass the credential as the action's `anthropic_api_key` input; the action sets it as `ANTHROPIC_API_KEY`, so it reaches the gateway in the `x-api-key` header.

For an `x-api-key` gateway, set the base URL in `env` and pass the gateway key as the input:

```yaml
env:
  ANTHROPIC_BASE_URL: https://llm-gateway.example.com

steps:
  - uses: anthropics/claude-code-action@v1
    with:
      anthropic_api_key: ${{ secrets.GATEWAY_API_KEY }}
```

For a bearer-token gateway, pass the same secret as both the `anthropic_api_key` input and `ANTHROPIC_AUTH_TOKEN` in the workflow `env` block. The action requires `anthropic_api_key`, `CLAUDE_CODE_OAUTH_TOKEN`, or workload identity federation before it launches Claude Code, and it doesn't read `ANTHROPIC_AUTH_TOKEN`, so the input satisfies that launch check while the env variable puts the key in the `Authorization` header the gateway reads. The copy in `x-api-key` is ignored:

```yaml
env:
  ANTHROPIC_BASE_URL: https://llm-gateway.example.com
  ANTHROPIC_AUTH_TOKEN: ${{ secrets.GATEWAY_API_KEY }}

steps:
  - uses: anthropics/claude-code-action@v1
    with:
      anthropic_api_key: ${{ secrets.GATEWAY_API_KEY }}
```

For the action's other authentication options, including `CLAUDE_CODE_OAUTH_TOKEN` and workload identity federation, see **Claude Code GitHub Actions** and the action's **README**.

Agent SDK

The **Agent SDK** has no gateway-specific options; it passes environment variables to the Claude Code process it spawns. Each SDK accepts an `env` option that sets the spawned process's environment, and the TypeScript and Python SDKs treat it differently:

- TypeScript: the spawned process inherits the parent environment by default, but setting `options.env` replaces the environment entirely. Spread `process.env` into it to keep your gateway variables.
- Python: `ClaudeAgentOptions(env=...)` merges on top of the inherited environment, so gateway variables set in the parent process carry through without spreading.

Slack, web, and Remote Control

Claude Code in Slack and **Claude Code on the web** are Anthropic-hosted products that always use Anthropic's API; they aren't part of a gateway deployment. Gateway variables set in a cloud session's environment configuration are not applied. If your traffic must stay on the gateway, don't enable these surfaces for those users.

Remote Control and **voice dictation** both rely on a claude.ai identity: Remote Control to pair a live session with your account, and voice dictation to reach the claude.ai transcription endpoint. They are unavailable while `ANTHROPIC_API_KEY`, `ANTHROPIC_AUTH_TOKEN`, or an `apiKeyHelper` is active. To use either, unset the gateway credential and log in with claude.ai instead; `/doctor` names the variable to unset.

Additional configuration

These settings cover cases beyond the base URL and credential. Set them only if your administrator's instructions or the **troubleshooting table** call for one.

Send additional headers

Some gateways route or tag requests using a custom header in addition to the credential, for example a tenant identifier or a routing key. To send one, set `ANTHROPIC_CUSTOM_HEADERS` with one `Name: Value` pair per line. The example below adds a routing header named `X-Org-Route`:

Bash or Zsh

```
export ANTHROPIC_BASE_URL=https://llm-gateway.example.com
export ANTHROPIC_AUTH_TOKEN=sk-gateway-key
```

PowerShell

```
$env:ANTHROPIC_BASE_URL = "https://llm-gateway.example.com"
$env:ANTHROPIC_AUTH_TOKEN = "sk-gateway-key"
```

```
curl -X POST "$ANTHROPIC_BASE_URL/v1/messages" \
  -H "Authorization: Bearer $ANTHROPIC_AUTH_TOKEN" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

```
Invoke-RestMethod -Method Post -Uri
"$env:ANTHROPIC_BASE_URL/v1/messages" `
  -Headers @{ "Authorization" = "Bearer
$env:ANTHROPIC_AUTH_TOKEN"; "anthropic-version" = "2023-06-01" }
`
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```
  const result = query({
    prompt: "...",
    options: {
      env: {
        ...process.env,
        ANTHROPIC_BASE_URL: "https://llm-gateway.example.com",
        ANTHROPIC_AUTH_TOKEN: process.env.GATEWAY_KEY,
      },
    },
  })
```

```
export ANTHROPIC_CUSTOM_HEADERS="X-Org-Route: prod"
```

```
$env:ANTHROPIC_CUSTOM_HEADERS = "X-Org-Route: prod"
```

For example, a script that reads from a vault:

```
#!/bin/bash
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference its path in `~/.claude/settings.json`:

```
{
  "apiKeyHelper": "~/bin/get-gateway-key.sh"
}
```

For example, a script that reads from a vault:

```
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference the PowerShell invocation in `%USERPROFILE%\.claude\settings.json`, escaping the backslashes in the JSON string:

```
{
  "apiKeyHelper": "powershell -NoProfile -File C:\\scripts\\get-
gateway-key.ps1"
}
```

```
export ANTHROPIC_BEDROCK_BASE_URL=https://llm-
gateway.example.com/bedrock
export CLAUDE_CODE_SKIP_BEDROCK_AUTH=1
export CLAUDE_CODE_USE_BEDROCK=1
```

```
$env:ANTHROPIC_BEDROCK_BASE_URL = "https://llm-
gateway.example.com/bedrock"
$env:CLAUDE_CODE_SKIP_BEDROCK_AUTH = "1"
$env:CLAUDE_CODE_USE_BEDROCK = "1"
```

```
export ANTHROPIC_VERTEX_BASE_URL=https://llm-
gateway.example.com/vertex
export ANTHROPIC_VERTEX_PROJECT_ID=your-gcp-project-id
export CLAUDE_CODE_SKIP_VERTEX_AUTH=1
export CLAUDE_CODE_USE_VERTEX=1
export CLOUD_ML_REGION=us-east5
```

```
$env:ANTHROPIC_VERTEX_BASE_URL = "https://llm-
gateway.example.com/vertex"
$env:ANTHROPIC_VERTEX_PROJECT_ID = "your-gcp-project-id"
$env:CLAUDE_CODE_SKIP_VERTEX_AUTH = "1"
$env:CLAUDE_CODE_USE_VERTEX = "1"
$env:CLOUD_ML_REGION = "us-east5"
```

```
export ANTHROPIC_FOUNDRY_BASE_URL=https://llm-
gateway.example.com/foundry
export ANTHROPIC_FOUNDRY_API_KEY=sk-gateway-key
export CLAUDE_CODE_USE_FOUNDRY=1
```

```
$env:ANTHROPIC_FOUNDRY_BASE_URL = "https://llm-
gateway.example.com/foundry"
$env:ANTHROPIC_FOUNDRY_API_KEY = "sk-gateway-key"
$env:CLAUDE_CODE_USE_FOUNDRY = "1"
```

```
export ANTHROPIC_AWS_BASE_URL=https://llm-
gateway.example.com/anthropic-aws
export ANTHROPIC_AWS_WORKSPACE_ID=wrkspc_01ABCDEFGHIJKLMN
export CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH=1
export CLAUDE_CODE_USE_ANTHROPIC_AWS=1
```

```
$env:ANTHROPIC_AWS_BASE_URL = "https://llm-
gateway.example.com/anthropic-aws"
$env:ANTHROPIC_AWS_WORKSPACE_ID = "wrkspc_01ABCDEFGHIJKLMN"
$env:CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH = "1"
$env:CLAUDE_CODE_USE_ANTHROPIC_AWS = "1"
```

You can also set `ANTHROPIC_CUSTOM_HEADERS` in the `env` block of a settings file. Use `\n` between pairs there, since JSON strings can't span multiple lines:

```
{
  "env": {
    "ANTHROPIC_CUSTOM_HEADERS": "X-Org-Route: prod\nX-Tenant:
acme"
  }
}
```

Add gateway models to the model picker

Model discovery queries the gateway for its model list at startup and adds those names to the `/model` picker alongside the built-in entries.

Enable it if your gateway serves model names that aren't in Claude Code's built-in list and you want to select them from the picker. If the built-in models are what you use, you don't need discovery; your administrator may also have already enabled it through managed settings.

To enable it, set `CLAUDE_CODE_ENABLE_GATEWAY_MODEL_DISCOVERY=1` in your shell or in the `env` block of `~/.claude/settings.json`. Discovery requires Claude Code v2.1.129 or later.

Discovered models appear as additional `/model` entries labeled `From gateway`. To confirm discovery ran, start `claude --debug` and look for the `[gatewayDiscovery]` lines: a success logs how many models were cached, and a `404`, timeout, or redirect is recorded there too. For when discovery runs, what it filters, and the response format gateways serve, see the **model discovery reference**.

Rotate credentials with apiKeyHelper

An `apiKeyHelper` is a command Claude Code runs to fetch your gateway credential, instead of reading it from a static environment variable.

Use a helper when the credential expires on a schedule, comes from a vault or SSO command, or your administrator told you to configure one. If your credential is a fixed string you set once, the **credential variable** is all you need and you can skip this section.

The helper is any shell command that prints the current credential to stdout. Claude Code runs it through your system shell, so on Windows it can be an executable or a PowerShell invocation. Write the script, make it executable, and reference it from `apiKeyHelper` in your **settings file**:

Bash or Zsh

```
export ANTHROPIC_BASE_URL=https://llm-gateway.example.com
export ANTHROPIC_AUTH_TOKEN=sk-gateway-key
```

PowerShell

```
$env:ANTHROPIC_BASE_URL = "https://llm-gateway.example.com"
$env:ANTHROPIC_AUTH_TOKEN = "sk-gateway-key"
```

```
curl -X POST "$ANTHROPIC_BASE_URL/v1/messages" \
  -H "Authorization: Bearer $ANTHROPIC_AUTH_TOKEN" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

```
Invoke-RestMethod -Method Post -Uri
"$env:ANTHROPIC_BASE_URL/v1/messages" `
  -Headers @{ "Authorization" = "Bearer
$env:ANTHROPIC_AUTH_TOKEN"; "anthropic-version" = "2023-06-01" }
`
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```
  const result = query({
    prompt: "...",
    options: {
      env: {
        ...process.env,
```

```
        ANTHROPIC_BASE_URL: "https://llm-gateway.example.com",
        ANTHROPIC_AUTH_TOKEN: process.env.GATEWAY_KEY,
      },
    },
  })
```

```
export ANTHROPIC_CUSTOM_HEADERS="X-Org-Route: prod"
```

```
$env:ANTHROPIC_CUSTOM_HEADERS = "X-Org-Route: prod"
```

For example, a script that reads from a vault:

```
#!/bin/bash
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference its path in `~/.claude/settings.json`:

```
{
  "apiKeyHelper": "~/bin/get-gateway-key.sh"
}
```

For example, a script that reads from a vault:

```
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference the PowerShell invocation in `%USERPROFILE%\.claude\settings.json`, escaping the backslashes in the JSON string:

```
{
  "apiKeyHelper": "powershell -NoProfile -File C:\\scripts\\get-
gateway-key.ps1"
}
```

```
export ANTHROPIC_BEDROCK_BASE_URL=https://llm-
gateway.example.com/bedrock
export CLAUDE_CODE_SKIP_BEDROCK_AUTH=1
export CLAUDE_CODE_USE_BEDROCK=1
```

```
$env:ANTHROPIC_BEDROCK_BASE_URL = "https://llm-
gateway.example.com/bedrock"
$env:CLAUDE_CODE_SKIP_BEDROCK_AUTH = "1"
$env:CLAUDE_CODE_USE_BEDROCK = "1"
```

```
export ANTHROPIC_VERTEX_BASE_URL=https://llm-
gateway.example.com/vertex
export ANTHROPIC_VERTEX_PROJECT_ID=your-gcp-project-id
export CLAUDE_CODE_SKIP_VERTEX_AUTH=1
export CLAUDE_CODE_USE_VERTEX=1
export CLOUD_ML_REGION=us-east5
```

```
$env:ANTHROPIC_VERTEX_BASE_URL = "https://llm-
gateway.example.com/vertex"
$env:ANTHROPIC_VERTEX_PROJECT_ID = "your-gcp-project-id"
$env:CLAUDE_CODE_SKIP_VERTEX_AUTH = "1"
$env:CLAUDE_CODE_USE_VERTEX = "1"
$env:CLOUD_ML_REGION = "us-east5"
```

```
export ANTHROPIC_FOUNDRY_BASE_URL=https://llm-
gateway.example.com/foundry
export ANTHROPIC_FOUNDRY_API_KEY=sk-gateway-key
export CLAUDE_CODE_USE_FOUNDRY=1
```

```
$env:ANTHROPIC_FOUNDRY_BASE_URL = "https://llm-
gateway.example.com/foundry"
$env:ANTHROPIC_FOUNDRY_API_KEY = "sk-gateway-key"
$env:CLAUDE_CODE_USE_FOUNDRY = "1"
```

```
export ANTHROPIC_AWS_BASE_URL=https://llm-
gateway.example.com/anthropic-aws
export ANTHROPIC_AWS_WORKSPACE_ID=wrkspc_01ABCDEFGHIJKLMN
export CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH=1
export CLAUDE_CODE_USE_ANTHROPIC_AWS=1
```

```
$env:ANTHROPIC_AWS_BASE_URL = "https://llm-
gateway.example.com/anthropic-aws"
$env:ANTHROPIC_AWS_WORKSPACE_ID = "wrkspc_01ABCDEFGHIJKLMN"
$env:CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH = "1"
$env:CLAUDE_CODE_USE_ANTHROPIC_AWS = "1"
```

Claude Code caches the helper's output for five minutes by default and re-runs it when a request returns HTTP 401. To change the cache lifetime, set `CLAUDE_CODE_API_KEY_HELPER_TTL_MS` in milliseconds, for example `CLAUDE_CODE_API_KEY_HELPER_TTL_MS=900000` for 15 minutes.

The helper's value is sent in both the `Authorization` and `x-api-key` headers, so it works whichever header your gateway reads.

Route to a cloud provider through a gateway

These configurations point Claude Code at a gateway through a provider-specific base URL variable in place of `ANTHROPIC_BASE_URL`. Bedrock and Vertex gateways accept those providers' native request formats; Foundry and Claude Platform on AWS gateways accept the Anthropic Messages format and differ only in which base URL variable reaches them.

Use one only if your gateway team specifically named Bedrock, Vertex, Foundry, or the Claude Platform on AWS. If the **verification request** above returned JSON, you can skip this section.

Set the block for the provider your gateway team named. The skip-auth variables tell Claude Code not to sign requests with provider credentials, since the gateway holds those. If the gateway needs its own token, add `ANTHROPIC_AUTH_TOKEN` after the block, except for Foundry, which uses `ANTHROPIC_FOUNDRY_API_KEY` as shown.

Amazon Bedrock

Bash or Zsh

```
export ANTHROPIC_BASE_URL=https://llm-gateway.example.com
export ANTHROPIC_AUTH_TOKEN=sk-gateway-key
```

PowerShell

```
$env:ANTHROPIC_BASE_URL = "https://llm-gateway.example.com"
$env:ANTHROPIC_AUTH_TOKEN = "sk-gateway-key"
```

```
curl -X POST "$ANTHROPIC_BASE_URL/v1/messages" \
  -H "Authorization: Bearer $ANTHROPIC_AUTH_TOKEN" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

```
Invoke-RestMethod -Method Post -Uri
"$env:ANTHROPIC_BASE_URL/v1/messages" `
  -Headers @{ "Authorization" = "Bearer
$env:ANTHROPIC_AUTH_TOKEN"; "anthropic-version" = "2023-06-01" }
`
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```
  const result = query({
    prompt: "...",
    options: {
      env: {
        ...process.env,
        ANTHROPIC_BASE_URL: "https://llm-gateway.example.com",
        ANTHROPIC_AUTH_TOKEN: process.env.GATEWAY_KEY,
      },
    },
  })
```

```
export ANTHROPIC_CUSTOM_HEADERS="X-Org-Route: prod"
```

```
$env:ANTHROPIC_CUSTOM_HEADERS = "X-Org-Route: prod"
```

For example, a script that reads from a vault:

```
#!/bin/bash
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference its path in `~/.claude/settings.json`:

```
{
  "apiKeyHelper": "~/bin/get-gateway-key.sh"
}
```

For example, a script that reads from a vault:

```
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference the PowerShell invocation in `%USERPROFILE%\.claude\settings.json`, escaping the backslashes in the JSON string:

```
{
  "apiKeyHelper": "powershell -NoProfile -File C:\\scripts\\get-
gateway-key.ps1"
}
```

```
export ANTHROPIC_BEDROCK_BASE_URL=https://llm-
gateway.example.com/bedrock
export CLAUDE_CODE_SKIP_BEDROCK_AUTH=1
export CLAUDE_CODE_USE_BEDROCK=1
```

```
$env:ANTHROPIC_BEDROCK_BASE_URL = "https://llm-
gateway.example.com/bedrock"
$env:CLAUDE_CODE_SKIP_BEDROCK_AUTH = "1"
$env:CLAUDE_CODE_USE_BEDROCK = "1"
```

```
export ANTHROPIC_VERTEX_BASE_URL=https://llm-
gateway.example.com/vertex
export ANTHROPIC_VERTEX_PROJECT_ID=your-gcp-project-id
export CLAUDE_CODE_SKIP_VERTEX_AUTH=1
export CLAUDE_CODE_USE_VERTEX=1
export CLOUD_ML_REGION=us-east5
```

```
$env:ANTHROPIC_VERTEX_BASE_URL = "https://llm-
gateway.example.com/vertex"
$env:ANTHROPIC_VERTEX_PROJECT_ID = "your-gcp-project-id"
$env:CLAUDE_CODE_SKIP_VERTEX_AUTH = "1"
$env:CLAUDE_CODE_USE_VERTEX = "1"
$env:CLOUD_ML_REGION = "us-east5"
```

```
export ANTHROPIC_FOUNDRY_BASE_URL=https://llm-
gateway.example.com/foundry
export ANTHROPIC_FOUNDRY_API_KEY=sk-gateway-key
export CLAUDE_CODE_USE_FOUNDRY=1
```

```
$env:ANTHROPIC_FOUNDRY_BASE_URL = "https://llm-
gateway.example.com/foundry"
$env:ANTHROPIC_FOUNDRY_API_KEY = "sk-gateway-key"
$env:CLAUDE_CODE_USE_FOUNDRY = "1"
```

```
export ANTHROPIC_AWS_BASE_URL=https://llm-
gateway.example.com/anthropic-aws
export ANTHROPIC_AWS_WORKSPACE_ID=wrkspc_01ABCDEFGHIJKLMN
export CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH=1
export CLAUDE_CODE_USE_ANTHROPIC_AWS=1
```

```
$env:ANTHROPIC_AWS_BASE_URL = "https://llm-
gateway.example.com/anthropic-aws"
$env:ANTHROPIC_AWS_WORKSPACE_ID = "wrkspc_01ABCDEFGHIJKLMN"
$env:CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH = "1"
$env:CLAUDE_CODE_USE_ANTHROPIC_AWS = "1"
```

Google Vertex AI

Bash or Zsh

```
export ANTHROPIC_BASE_URL=https://llm-gateway.example.com
export ANTHROPIC_AUTH_TOKEN=sk-gateway-key
```

PowerShell

```
$env:ANTHROPIC_BASE_URL = "https://llm-gateway.example.com"
$env:ANTHROPIC_AUTH_TOKEN = "sk-gateway-key"
```

```
curl -X POST "$ANTHROPIC_BASE_URL/v1/messages" \
  -H "Authorization: Bearer $ANTHROPIC_AUTH_TOKEN" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

```
Invoke-RestMethod -Method Post -Uri
"$env:ANTHROPIC_BASE_URL/v1/messages" `
  -Headers @{ "Authorization" = "Bearer
$env:ANTHROPIC_AUTH_TOKEN"; "anthropic-version" = "2023-06-01" }
`
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```
  const result = query({
    prompt: "...",
    options: {
      env: {
        ...process.env,
        ANTHROPIC_BASE_URL: "https://llm-gateway.example.com",
        ANTHROPIC_AUTH_TOKEN: process.env.GATEWAY_KEY,
      },
    },
  })
```

```
export ANTHROPIC_CUSTOM_HEADERS="X-Org-Route: prod"
```

```
$env:ANTHROPIC_CUSTOM_HEADERS = "X-Org-Route: prod"
```

For example, a script that reads from a vault:

```
#!/bin/bash
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference its path in `~/.claude/settings.json`:

```
{
  "apiKeyHelper": "~/bin/get-gateway-key.sh"
}
```

For example, a script that reads from a vault:

```
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference the PowerShell invocation in `%USERPROFILE%\.claude\settings.json`, escaping the backslashes in the JSON string:

```
{
  "apiKeyHelper": "powershell -NoProfile -File C:\\scripts\\get-
gateway-key.ps1"
}
```

```
export ANTHROPIC_BEDROCK_BASE_URL=https://llm-
gateway.example.com/bedrock
export CLAUDE_CODE_SKIP_BEDROCK_AUTH=1
export CLAUDE_CODE_USE_BEDROCK=1
```

```
$env:ANTHROPIC_BEDROCK_BASE_URL = "https://llm-
gateway.example.com/bedrock"
$env:CLAUDE_CODE_SKIP_BEDROCK_AUTH = "1"
$env:CLAUDE_CODE_USE_BEDROCK = "1"
```

```
export ANTHROPIC_VERTEX_BASE_URL=https://llm-
gateway.example.com/vertex
export ANTHROPIC_VERTEX_PROJECT_ID=your-gcp-project-id
export CLAUDE_CODE_SKIP_VERTEX_AUTH=1
export CLAUDE_CODE_USE_VERTEX=1
export CLOUD_ML_REGION=us-east5
```

```
$env:ANTHROPIC_VERTEX_BASE_URL = "https://llm-
gateway.example.com/vertex"
$env:ANTHROPIC_VERTEX_PROJECT_ID = "your-gcp-project-id"
$env:CLAUDE_CODE_SKIP_VERTEX_AUTH = "1"
$env:CLAUDE_CODE_USE_VERTEX = "1"
$env:CLOUD_ML_REGION = "us-east5"
```

```
export ANTHROPIC_FOUNDRY_BASE_URL=https://llm-
gateway.example.com/foundry
export ANTHROPIC_FOUNDRY_API_KEY=sk-gateway-key
export CLAUDE_CODE_USE_FOUNDRY=1
```

```
$env:ANTHROPIC_FOUNDRY_BASE_URL = "https://llm-
gateway.example.com/foundry"
$env:ANTHROPIC_FOUNDRY_API_KEY = "sk-gateway-key"
$env:CLAUDE_CODE_USE_FOUNDRY = "1"
```

```
export ANTHROPIC_AWS_BASE_URL=https://llm-
gateway.example.com/anthropic-aws
export ANTHROPIC_AWS_WORKSPACE_ID=wrkspc_01ABCDEFGHIJKLMN
export CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH=1
export CLAUDE_CODE_USE_ANTHROPIC_AWS=1
```

```
$env:ANTHROPIC_AWS_BASE_URL = "https://llm-
gateway.example.com/anthropic-aws"
$env:ANTHROPIC_AWS_WORKSPACE_ID = "wrkspc_01ABCDEFGHIJKLMN"
$env:CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH = "1"
$env:CLAUDE_CODE_USE_ANTHROPIC_AWS = "1"
```

Microsoft Foundry

Put the gateway's credential in `ANTHROPIC_FOUNDRY_API_KEY`; it is sent to the gateway as the `x-`

`api-key` header. `CLAUDE_CODE_SKIP_FOUNDRY_AUTH` doesn't apply here: without an API key, the Foundry client fails every request before it leaves the machine.

Bash or Zsh

```
export ANTHROPIC_BASE_URL=https://llm-gateway.example.com
export ANTHROPIC_AUTH_TOKEN=sk-gateway-key
```

PowerShell

```
$env:ANTHROPIC_BASE_URL = "https://llm-gateway.example.com"
$env:ANTHROPIC_AUTH_TOKEN = "sk-gateway-key"
```

```
curl -X POST "$ANTHROPIC_BASE_URL/v1/messages" \
  -H "Authorization: Bearer $ANTHROPIC_AUTH_TOKEN" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

```
Invoke-RestMethod -Method Post -Uri
"$env:ANTHROPIC_BASE_URL/v1/messages" `
  -Headers @{ "Authorization" = "Bearer
$env:ANTHROPIC_AUTH_TOKEN"; "anthropic-version" = "2023-06-01" }
`
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```
  const result = query({
    prompt: "...",
    options: {
      env: {
        ...process.env,
        ANTHROPIC_BASE_URL: "https://llm-gateway.example.com",
        ANTHROPIC_AUTH_TOKEN: process.env.GATEWAY_KEY,
      },
    },
  })
```

```
export ANTHROPIC_CUSTOM_HEADERS="X-Org-Route: prod"
```

```
$env:ANTHROPIC_CUSTOM_HEADERS = "X-Org-Route: prod"
```

For example, a script that reads from a vault:

```
#!/bin/bash
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference its path in `~/.claude/settings.json`:

```
{
  "apiKeyHelper": "~/bin/get-gateway-key.sh"
}
```

For example, a script that reads from a vault:

```
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference the PowerShell invocation in `%USERPROFILE%\.claude\settings.json`, escaping the backslashes in the JSON string:

```
{
  "apiKeyHelper": "powershell -NoProfile -File C:\\scripts\\get-
gateway-key.ps1"
}
```

```
export ANTHROPIC_BEDROCK_BASE_URL=https://llm-
gateway.example.com/bedrock
export CLAUDE_CODE_SKIP_BEDROCK_AUTH=1
export CLAUDE_CODE_USE_BEDROCK=1
```

```
$env:ANTHROPIC_BEDROCK_BASE_URL = "https://llm-
gateway.example.com/bedrock"
$env:CLAUDE_CODE_SKIP_BEDROCK_AUTH = "1"
$env:CLAUDE_CODE_USE_BEDROCK = "1"
```

```
export ANTHROPIC_VERTEX_BASE_URL=https://llm-
gateway.example.com/vertex
export ANTHROPIC_VERTEX_PROJECT_ID=your-gcp-project-id
export CLAUDE_CODE_SKIP_VERTEX_AUTH=1
export CLAUDE_CODE_USE_VERTEX=1
export CLOUD_ML_REGION=us-east5
```

```
$env:ANTHROPIC_VERTEX_BASE_URL = "https://llm-
gateway.example.com/vertex"
$env:ANTHROPIC_VERTEX_PROJECT_ID = "your-gcp-project-id"
$env:CLAUDE_CODE_SKIP_VERTEX_AUTH = "1"
$env:CLAUDE_CODE_USE_VERTEX = "1"
$env:CLOUD_ML_REGION = "us-east5"
```

```
export ANTHROPIC_FOUNDRY_BASE_URL=https://llm-
gateway.example.com/foundry
export ANTHROPIC_FOUNDRY_API_KEY=sk-gateway-key
export CLAUDE_CODE_USE_FOUNDRY=1
```

```
$env:ANTHROPIC_FOUNDRY_BASE_URL = "https://llm-
gateway.example.com/foundry"
$env:ANTHROPIC_FOUNDRY_API_KEY = "sk-gateway-key"
$env:CLAUDE_CODE_USE_FOUNDRY = "1"
```

```
export ANTHROPIC_AWS_BASE_URL=https://llm-
gateway.example.com/anthropic-aws
export ANTHROPIC_AWS_WORKSPACE_ID=wrkspc_01ABCDEFGHIJKLMN
export CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH=1
export CLAUDE_CODE_USE_ANTHROPIC_AWS=1
```

```
$env:ANTHROPIC_AWS_BASE_URL = "https://llm-
gateway.example.com/anthropic-aws"
$env:ANTHROPIC_AWS_WORKSPACE_ID = "wrkspc_01ABCDEFGHIJKLMN"
$env:CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH = "1"
$env:CLAUDE_CODE_USE_ANTHROPIC_AWS = "1"
```

Claude Platform on AWS

See **Claude Platform on AWS** for the workspace ID.

Bash or Zsh

```
export ANTHROPIC_BASE_URL=https://llm-gateway.example.com
export ANTHROPIC_AUTH_TOKEN=sk-gateway-key
```

PowerShell

```
$env:ANTHROPIC_BASE_URL = "https://llm-gateway.example.com"
$env:ANTHROPIC_AUTH_TOKEN = "sk-gateway-key"
```

```
curl -X POST "$ANTHROPIC_BASE_URL/v1/messages" \
  -H "Authorization: Bearer $ANTHROPIC_AUTH_TOKEN" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

```
Invoke-RestMethod -Method Post -Uri
"$env:ANTHROPIC_BASE_URL/v1/messages" `
  -Headers @{ "Authorization" = "Bearer
$env:ANTHROPIC_AUTH_TOKEN"; "anthropic-version" = "2023-06-01" }
`
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```
const result = query({
  prompt: "...",
  options: {
    env: {
      ...process.env,
      ANTHROPIC_BASE_URL: "https://llm-gateway.example.com",
      ANTHROPIC_AUTH_TOKEN: process.env.GATEWAY_KEY,
    },
  },
})
```

```
export ANTHROPIC_CUSTOM_HEADERS="X-Org-Route: prod"
```

```
$env:ANTHROPIC_CUSTOM_HEADERS = "X-Org-Route: prod"
```

For example, a script that reads from a vault:

```
#!/bin/bash
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference its path in `~/.claude/settings.json`:

```
{
  "apiKeyHelper": "~/bin/get-gateway-key.sh"
}
```

For example, a script that reads from a vault:

```
vault kv get -field=api_key secret/llm-gateway/claude-code
```

Reference the PowerShell invocation in `%USERPROFILE%\.claude\settings.json`, escaping the backslashes in the JSON string:

```
{
  "apiKeyHelper": "powershell -NoProfile -File C:\\scripts\\get-
gateway-key.ps1"
}
```

```
export ANTHROPIC_BEDROCK_BASE_URL=https://llm-
gateway.example.com/bedrock
export CLAUDE_CODE_SKIP_BEDROCK_AUTH=1
export CLAUDE_CODE_USE_BEDROCK=1
```

```
$env:ANTHROPIC_BEDROCK_BASE_URL = "https://llm-
gateway.example.com/bedrock"
$env:CLAUDE_CODE_SKIP_BEDROCK_AUTH = "1"
$env:CLAUDE_CODE_USE_BEDROCK = "1"
```

```
export ANTHROPIC_VERTEX_BASE_URL=https://llm-
gateway.example.com/vertex
export ANTHROPIC_VERTEX_PROJECT_ID=your-gcp-project-id
export CLAUDE_CODE_SKIP_VERTEX_AUTH=1
export CLAUDE_CODE_USE_VERTEX=1
export CLOUD_ML_REGION=us-east5
```

```
$env:ANTHROPIC_VERTEX_BASE_URL = "https://llm-
gateway.example.com/vertex"
$env:ANTHROPIC_VERTEX_PROJECT_ID = "your-gcp-project-id"
$env:CLAUDE_CODE_SKIP_VERTEX_AUTH = "1"
$env:CLAUDE_CODE_USE_VERTEX = "1"
$env:CLOUD_ML_REGION = "us-east5"
```

```
export ANTHROPIC_FOUNDRY_BASE_URL=https://llm-
gateway.example.com/foundry
export ANTHROPIC_FOUNDRY_API_KEY=sk-gateway-key
export CLAUDE_CODE_USE_FOUNDRY=1
```

```
$env:ANTHROPIC_FOUNDRY_BASE_URL = "https://llm-
gateway.example.com/foundry"
$env:ANTHROPIC_FOUNDRY_API_KEY = "sk-gateway-key"
$env:CLAUDE_CODE_USE_FOUNDRY = "1"
```

```
export ANTHROPIC_AWS_BASE_URL=https://llm-
gateway.example.com/anthropic-aws
export ANTHROPIC_AWS_WORKSPACE_ID=wrkspc_01ABCDEFGHIJKLMN
export CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH=1
export CLAUDE_CODE_USE_ANTHROPIC_AWS=1
```

```
$env:ANTHROPIC_AWS_BASE_URL = "https://llm-
gateway.example.com/anthropic-aws"
$env:ANTHROPIC_AWS_WORKSPACE_ID = "wrkspc_01ABCDEFGHIJKLMN"
$env:CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH = "1"
$env:CLAUDE_CODE_USE_ANTHROPIC_AWS = "1"
```

Troubleshoot gateway errors

These are the most common errors when running Claude Code through a gateway, with the gateway-side cause and the fix:

Error	Cause	Fix
A startup warning naming two credential sources and ending in `auth may not work as expected`. Older versions show `Auth conflict: Both a token (SOURCE) and an API key (SOURCE) are set`	A gateway credential and a saved login are both active; the variable is used for requests, but the stale login can cause unexpected auth behavior	Unset the variable to use the saved login, or run `/logout` to use the gateway credential

Error	Cause	Fix
instead.		
`401` errors naming an invalid or unrecognized token	The credential isn't one the gateway issued, or it's in a header the gateway doesn't read	Confirm the variable matches your credential kind in the **credential table**, and regenerate the key at the gateway if it was revoked
`Unable to connect to API (ConnectionRefused)`, or `(ECONNREFUSED)` from npm installs, often after a silent pause while Claude Code **retries with backoff**	Nothing answered at the base URL: the address is wrong, or a VPN or firewall blocks the path to the gateway	Run the **curl test above**, which fails immediately with the same cause, and confirm the URL and network path with your gateway team
`API returned an empty or malformed response (HTTP 200)`	The gateway or an intermediate proxy returned a non-API response, often an HTML error or login page	Test with the **curl request above**; fix the gateway route that returns non-JSON
`400` errors naming `context_management`, `Extra inputs are not permitted`, or other unrecognized fields	The gateway forwards requests to an upstream that rejects fields Claude Code sends to Anthropic-format endpoints	Set `CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS=1`, which suppresses most pre-release fields; see **feature pass-through**. Some betas aren't gated by this flag; for those, set the matching `CLAUDE_CODE_USE_*` provider variable so Claude Code sends only what that provider accepts
`400` errors naming `thinking` or `adaptive`, such as `Input tag 'adaptive' found`	The upstream model build doesn't accept adaptive reasoning, which Claude Code requests for Claude 4.6 and later models	Upgrade the gateway's upstream. On Opus 4.6 and Sonnet 4.6, `CLAUDE_CODE_DISABLE_ADAPTIVE_THINKING=1` works instead. The **model configuration** capability variables apply only to the provider configurations, such as `CLAUDE_CODE_USE_BEDROCK` and `CLAUDE_CODE_USE_VERTEX`, not behind an `ANTHROPIC_BASE_URL` gateway
`400` errors stating a context or token limit in the gateway's own words, such as `ContextWindowExceededError` or `prompt token count of N exceeds the limit of M`	The gateway enforces a smaller context than the model's native window and rewrites the upstream error, so the automatic compact-and-retry, which matches Anthropic's `prompt is too long` wording, doesn't fire	Run `/compact` to recover the session. To prevent it, set `CLAUDE_CODE_AUTO_COMPACT_WINDOW` to the gateway's limit; the value is clamped to at least 100,000 tokens and at most the model's context window, so a gateway limit below 100,000 can't be matched and `/compact` remains the recovery there. Also set `CLAUDE_CODE_MAX_OUTPUT_TOKENS` below the gateway model's output limit
Models missing from the `/model` picker	Gateway model names aren't in Claude Code's built-in list	Enable **gateway model discovery** or add names with the **model configuration** variables
Claude Code asks you to log in	The CLI has no credential of its	Set `ANTHROPIC_AUTH_TOKEN` somewhere

Error	Cause	Fix
even though the **curl test** succeeds	own: a reachable base URL isn't one, and an `env` block in a project's `.claude/settings.json` or `.claude/settings.local.json` applies only after the first-run wizard and trust prompt	Claude Code reads before first-run setup: a shell export, the `env` block in `~/.claude/settings.json`, or managed settings
`ANTHROPIC_API_KEY` is set but ignored, with no prompt	The key needs a one-time approval in interactive sessions, and a previously declined key is ignored without asking again	Enable it under `/config` with the `Use custom API key` option
`This machine's managed settings require a first-party login`	Managed settings include `forceLoginMethod` or `forceLoginOrgUUID`, which on Claude Code v2.1.146 and later cannot coexist with `ANTHROPIC_API_KEY`, `ANTHROPIC_AUTH_TOKEN`, or `apiKeyHelper`	Your administrator must remove `forceLoginMethod` and `forceLoginOrgUUID` from managed settings to use gateway credentials, or remove the gateway credential to use first-party login. The two cannot be combined
`403` with an HTML body such as `403 Forbidden`, when the gateway's own logs show no request received	A web application firewall or reverse proxy in front of the gateway blocked the request body before it reached the gateway. Claude Code prompts include XML-style tags and source code that match cross-site-scripting body rules, so a short curl test passes while a real session doesn't	Exempt the gateway's `/v1/messages` path from request-body inspection. On AWS WAF this is the `CrossSiteScripting_Body` managed rule; on nginx with ModSecurity it is the equivalent OWASP CRS body rules
Certificate or TLS errors such as `SSL certificate verification failed` or `Self-signed certificate detected`, when the **curl test** succeeds	Claude Code's runtime isn't trusting the same certificate authority that `curl` uses. Common behind corporate TLS-inspection proxies	Set `NODE_EXTRA_CA_CERTS` to the CA bundle path; see **CA certificate store**

If Claude Code prompts you to log in repeatedly after removing gateway configuration, the cause is usually credential storage rather than the gateway; see **authentication errors**.

Related resources

- **LLM gateways overview**: what a gateway is and how it interacts with claude.ai subscriptions
- **Roll out an LLM gateway for your organization**: the admin-facing checklist for deploying and distributing gateway configuration
- **Gateway protocol reference**: what Claude Code sends to a gateway, including the headers and

fields the gateway must forward

- **Settings**: where settings files live and how the `env` block is read
- **Authentication**: how credential variables, `apiKeyHelper`, and OAuth login interact